Programmatoid and neuronoid computations

From a discussion with Nicolas Rougier et al

June 28, 2026

What about ?

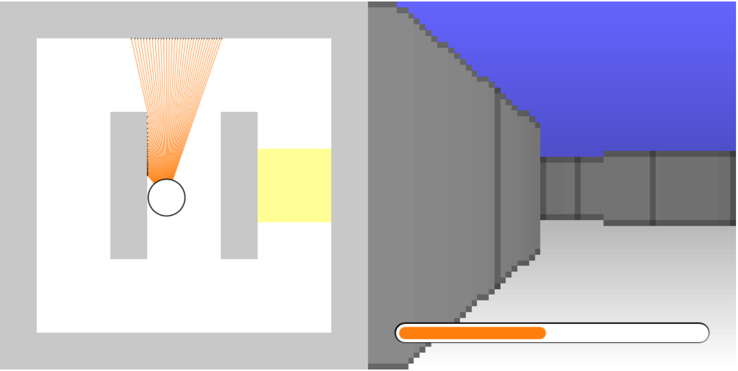
Programming (not training) neural networks

- With a benchmark and ...
- Using computer algebra.

But . . . why ?

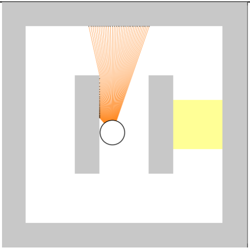
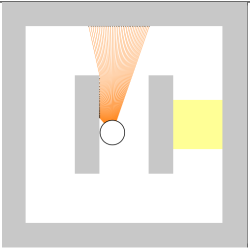
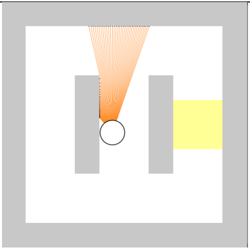
- For the fun :)
- To illustrate a feed-forward neural network computational property.
- To provide an approximate neural network complexity lower bound, given a task.
- To obtain a parsimonious computation tool.

The braincraft benchmark

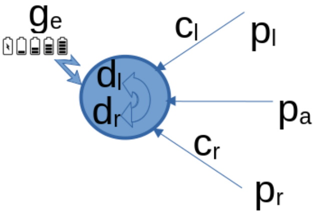


Documented in the README ☑ (please).

Three tasks:

		
Find the energy source leftward or rightward ☑.	Find the correct direction, given the blue cue ☑.	Choose the best energy source ☑.

A (simplified) bot, with:



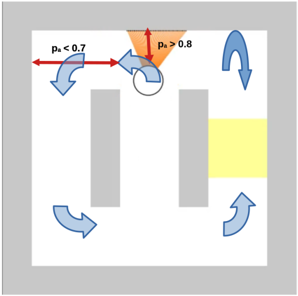
- left, ahead and right proximity sensors

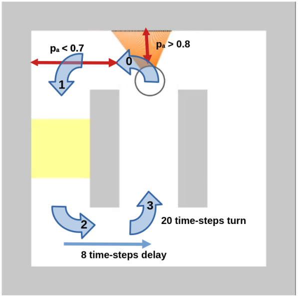
$$p_\bullet \in]0_{\text{no-wall}}, 1_{\text{wall-hit}}], \bullet \in \{l, a, r\},$$

- color sensors $c_{\bullet\dagger}$,
- energy gauge $g_e \in [0_{\text{death}}, 1_{\text{full}}]$,
- running at constant velocity, and

-turning leftward or rightward $d_l \in [0, 1_{+5^o}]$.

Heuristics:





- Stabilizes the forward motion using linear feedback correction.
- Performs quarter-turn using ahead proximity thresholds.
- Performs about-turn or half-corridor turn using delays.
- Stores the preferred quarter-turn direction, e.g. from cue.
- Stores previous energy feeds for comparison.

Programmatic implementation:

```python
class State:
    """ Implements a programmatoid state.
    """

    data = dict()
    """ The state data: dictionary.
        - Stores all input, output, and internal variables, including parameters.
    """

    def init(self):
        """ Inits some data, if needed.
        """

    def update(self):
        """ Updates, at each step, the data from input and sets the output value.
        """
```

- About 8 internal variables
  (forward/turns states, turn direction, delays, gauge values)
- About 40 lines of straight-line program code
  (beyond initialization and declarations)
  - when using about-turn ☑ and
  - about 50 when using quarter-turns only ☑.

So what ?

From programmatic
to programmatoid and neuronoid
implementation.

The notion of "programmatoid computation":

- It is an input-output straight-line program ☑ implementing an operator on either
  - binary values $\in \{0, 1\}$
  - or continuous normalized values $\in [0, 1]$,

which evolution is defined using an LN-system with linear units or the step-function $\mathrm{H}(x) \in \{0_{x<0}, 1/2_{x=0}, 1_{x>0}\}$ as non-linearity:

$$
\begin{array}{rcl}
o_n[t] & = & \left\{ \begin{array}{r} \mathrm{H}(z_n[t]) \\ z_n[t] \end{array} \right. , \\
z_n[t] & \overset{\mathrm{def}}{=} & \sum_{m \in \{1,M\}} w_{nm}\, i_m[t-1] + w_{n0}, \, n \in \{1, N\}
\end{array}
$$

where $i_m[t]$ is the m-th input at a discrete t, and $o_n[t]$ an output.

- The key idea:

programmatoid implements conditional expressions

on binary or numerical values:

```
value = if condition then true_value else false_value
```

$\longrightarrow$

```
value = H(condition) true_value + H(not condition) false_value
```

For a property $\mathcal{P}$, $\mathrm{H}(\mathcal{P}) \in \{0_{\mathcal{P} \text{ is false}}, 1_{\mathcal{P} \text{ is true}}\}$.

The notion of “sugared straight-line program”:

- It is an input-output piece of code with
 - instruction sequences, assigments, conditional instructions, and iterations (no general loops);
 - using [generalized] programmatoid computation in assignment.

- This implements the set of primitive recursive functions □.
- Any primitive recursive functions rewrites as a generalized programmatoid:
  - A generalized programmatoid computation includes:
    - exponential and logarithm as non-linearity, when not
    - other p □ure function.

The reasons why (not a formal proof):

- Iterations unfolding

```
for n from 0 to n_1 ≤ n_max do
   instructions_1 endfor

⟶

if n ≥ 0 then
   substitute(n = 0, instructions_1)
...
if n = n_1 then
   substitute(n = n_1, instructions_1)
```

- Conditional expansion

```
if condition_1 then instruction_1 instructions_2
elif condition_2 then instructions_3
...
else instructions_4 endif

⟶

if condition_1 then instruction_1 endif
if condition_1 then instructions_2 endif
if not condition_1 and condition_2 then instructions_3 endif
...
if not condition_1 and not condition_2 then instructions_4 endif
```

- Conditional reduction

```
if condition then name = value endif

⟶

name = if condition then value else name
```

- Introducing $\log(\cdot)$ and $\exp(\cdot)$ makes products, thus polynomials, implementable.
- Introducing polynomial, makes regular functions approximatable.

The notion of "neuronoid":

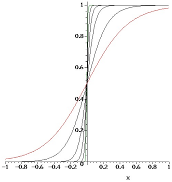
This names the not very biologically plausible simplest biological neuron inspired by mean-field modeling of the Hodgkin–Huxley neuronal axon model:

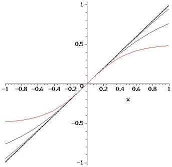
$$\tau \frac{\partial v_i}{\partial t}(t) + v_i(t) = z_i(t), z_i(t) \stackrel{\text{def}}{=} \mathrm{h}\left(\sum_j w_{ij}\, v_j(t) + w_{i0}\right),$$

using the normalized sigmoid with:

$$\mathrm{h}(-\infty) = 0, \mathrm{h}(0) = 1/2, \mathrm{h}'(0) = 1, \mathrm{h}(+\infty) = 1, \mathrm{h}(x) = 1 - \mathrm{h}(-x).$$

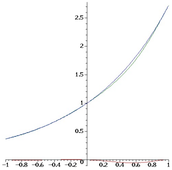
- A neuronoid $z \to h_{\tau=0}(\omega\, z)|_{\omega>10}$ approximates $H(\cdot)$ below 10^{-10}.
- A neuronoid $z \to \omega\, h_{\tau=0}(\frac{z}{\omega})\big|_{\omega>100}$ approximates $z \to z$ below 10^{-3}.



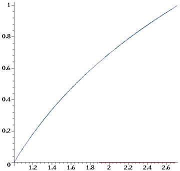


Further use of “programmatoid/neuronoid”:

Combinations of neuronoids $\tau = 0$ approximate regular functions:



The $\exp(\cdot)$ function below 10^{-3}



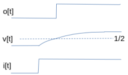
The $\log(\cdot)$ function below 10^{-2}

$$o \stackrel{\text{def}}{=} g \underbrace{\left(\frac{1}{K}\sum_k i_k\right)}_{mean} + (1-g) \underbrace{\left(\sum_k b_k\, i_k\right)}_{max}$$

for b_k based on $H(i_h > i_l)$ combinations.

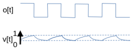
The softmax operator

Neuronoids with $\tau > 0$ implements temporal operators:

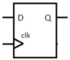


Delay operator

Bistable operator



Astable operator



clk	D	Q
0	0	Q
0	1	Q
1	0	0
1	1	1

D latch

Implemented functions 1/3

For binary variables `b_•` $\in \{0,1\}$, continuous variables `v_•` $\in [-1,1]$, and the sigmoid $h(\cdot)$ the following functions, defined previously, are implemented:

`v_o = h(v)`	The normalized sigmoid.
`v_o = H(v)`	Generalized step-function. `v_o` $\in \{0_{v<0}, 1/2_{v=0}, 1_{v>0}\}$ Converted to `h()` when the option `prgm["neuronoid"] = true`.
`v_o = Id(v)`	Identity function Converted to a neuronoid when the option `prgm["neuronoid"] = true`.
`b_o = And(b_1, ...)`	Conjunction with a variable number of binary arguments. Implements `b_1 and b_2 ...`
`b_o = Or(b_1, ...)`	Disjunction with a variable number of binary arguments. Implements `b_1 or b_2 ...`
`b_o = Not(b)`	Negation of a binary value. Implements `not b`
`b_ o = If_b(b_1, b'_1, ..., b'_0)`	Binary conditional expression, Implements `if b_1 then b'_1 ....`
`v_o = If_v(b_1, v_1, ..., v_0)`	Valued conditional expression, Implements `if b_1 then v_1 ....`
`v_o = Exp_v(v_1)`	Exponential function approximation Implements $v_1 \in [-1,1], e^{v_1} \in [1/e, e] \simeq [0.368, 2.718]$
`v_o = Log_v(v_1)`	Natural logarithm approximation Implements $v_1 \in [1,e], \log(v_1) \in [0,1]$
`v_o = Prod_v(v_1, v_2)`	Multiplication approximation Implements $v_i \in [-1,1], i \in \{1,2\}, v_1\, v_2 \in [-1,1]$
`v_o = Prod_b(b_1, v_1, ...)`	Sum of binary weights values Implements `b_1  v_1 + ....` Default missing value is 0.

Implemented functions 2/3

`v_o = Softmax(v_1, ..., G)`	Mean-max operator. `G` $\in [0_{maximum}, 1_{average}]$ controls the mean-max balance.
`Latch_b(b_o, b_i, b_c)`	Binary memory latch. `b_o` is the output, `b_i` is the input, `b_c` $\in \{0_{open}, 1_{latched}\}$ is the control.
`Latch_v(b_o, v_i, b_c)`	Valued memory latch. `b_o` is the output, `v_i` is the input, `b_c` $\in \{0_{open}, 1_{latched}\}$ is the control.
`Bistable(b_o, b_1, b_0)`	Two inputs bi-stable latch. `b_o` is the output, `b_0` resets to 0, `b_1` sets to 1.
`Bistable(b_o, b_i)`	One input bi-stable latch. `b_o` is the output, `b_i` is the two-way input.
`Spikeup(b_o, b_i)`	Spike generation on input rising front. `b_i` is the input, which rising front is detected.

Implemented functions 3/3

`Delay(b_o, b_i, T)`	Basic delayed output. `T` >0 is the delay, in number of global clock event, `b_o` is the output, `b_i` is the input, that must remains to 1.
`Delay_0(b_o, b_1, b_0, T)`	Delayed output. `T` >0 is the delay, in number of global clock event, `b_o` is the output, `b_1` is the input, `b_0` is the optional reset.
`Delay_1(b_o, b_1, b_0, T)`	Time bounded output. `T` >0 is the delay, in number of global clock event, `b_o` is the output, `b_1` is the input, `b_0` is the optional reset.
`Oscillator(b_o, b_c, T)`	Oscillatory output. `T` >0 is the period, in number of global clock event. `b_o` is the output, `b_c` $\in \{0_{stop}, 1_{run}\}$ is the control.

The programmatoid to neuronoid translator:

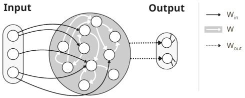
```
# Task1,2,3 implementation as a programmatoid
subs(
## Constants values
  gamma  = 0.035,   ### Linear feedback gain in forward mode
  alpha  = 5,            ### Maximal turn angle
  beta_1 = 0.8,          ### Proximity turn trigger threshold
  beta_0 = 0.7,          ### Proximity turn stop threshold
  delta  = 40,           ### About-turn number of steps
[
  prgm_options = { challenge = 0, networking = false },
  # Bot state control
  b_d = If_b(And(challenge = 2, Not(b_k), c_lb), 1, b_d),
  b_k = If_b(Or(challenge != 2 or c_lb or c_rb), 1, b_k),
  b_1 = And(Not(b_t) , p_a > 0.8),
  b_a = If_b(If_b(challenge = 3, And(g_d1 > 0, g_e - g_e1  > g_d1),
                               And(g_e1 > 0, g_e > g_e1)), 1, b_a),
  b_d = If_b(And(b_1, b_a), Not(b_d), b_d),
  Delay_0(b_w, And(b_1, b_a), delta),
  b_0 = And(b_t, If_b(b_a, b_w, p_a < 0.7)),
  b_t = If_b(b_1, 1, b_0, 0, b_t),
  b_s = And(Not(b_1),  b_0),
  ### Energy variables update
  g_d1 = If_v(And(b_s, g_e1 > 0, g_e > g_e1), g_e - g_e1, g_d1),
  g_e1 = If_v(b_s, g_e, g_e1),
  ### Navigation equations
  d_l = If_v(b_t, If_v(b_d, 5, 0), 0.035 * p_r),
  d_r = If_v(b_t, If_v(b_d, 0, 4), 0.035 * p_l)
]):
```

```
class State:
    """ Implements a programmatoid state.
    """

    data = dict()
    """ The state data: dictionary.
         - Stores all input, output, and internal variables, including parameters.
    """

    def init(self):
        """ Inits some data, if needed.
        """

    def update(self):
        """ Updates, at each step, the data from input and sets the output value.
        """
```



- From programmatoid equations ...
  - to Python code ... ... ▲

    a straight-line program with $h(\cdot)$ or $H(\cdot)$ non-linearities, if any.
  - to neural-network weights values ... ... ▲

    using $h(\cdot)$ non-linearities for all units.

The input syntax uses Maple ☑ syntax and the translator ☑ is implemented in this language, since based on computer algebra symbolic computing.

Open science approach and experimental reproducibility:

- Everything is available online
  under the CeCILL-C ☑ (for the code) and
  under CC-BY ☑ (for other resources) licenses.
- All sources are available in a `git` ☑ repository.
- All results are available via a single web-page ☑.
- All commands are gathered in fully documented `makefile` ☑:
  - The reproducibility ☑ test, simply writes:
    `make rebuild`.
  - The test's screen casts are recorded.
  - The complete process trace is made available,
    in wJSON ☑, easily readable.


That's
Y'all
Folk